

Studiu prin simulare în R

Sîrghe Matei-Ştefan

Popa Mircea

Bujor Ştefan

Afton Ungureanu Robert

26 May 2026

Cuprins

Documentatie Problema 1	2
Cerinta obligatorie	2
Cerinta 1	2
Cerinta 2	2
Cerinta 3	3
Cerinta 4	3
Cerinta 5	5
Cerinta 6	6
Cerinta 7	10
Cerinta 8	12
Cerinta ulterioara	12
Cerinta 1	12
Cerinta 2	13
Cerinta 3	14
Documentatie Problema 2	15
Cerinta obligatorie	15
Cerinta 1	15
Cerinta 2	15
Cerinta 3	17
Cerinta 4	18
Cerinta 5	18
Cerinta 6	19
Cerinta 7	19
Cerinta 8	19
Componentă bidimensională obligatorie	20
Cerinta 1	20
Cerinta 2	20
Cerinta 3	20
Cerinta 4	20
Cerinta 5	20
Cerinta 6	20

Documentatie Problema 1

In aceasta documentatie vom explica cum am rezolvat cerintele si deciziile luate cand vine vorba de rezolvarea acestora

Cerinta obligatorie

Cerinta 1

Am decis sa folosim un dataframe pentru a stoca toate datele care arata astfel:

```
date <- data.frame(  
  zi          = integer(),  
  total       = integer(),  
  normale     = integer(),  
  sus         = integer(),  
  verificate_s = integer(),  
  verificate_a = integer(),  
  verificate_a2 = integer(),  
  detectate_s  = integer(),  
  nedetectate_s = integer(),  
  detectate_a  = integer(),  
  nedetectate_a = integer(),  
  detectate_a2 = integer(),  
  nedetectate_a2 = integer()  
)
```

Cerinta 2

Putem considera ca numarul total de accesuri per zi urmaresc repartitia Poisson, deoarece modelul nostru matematic se bazeaza pe evenimente independente ce se intampla si sunt numarabile intr-un interval fix, avand adesea o rata medie de aparitie pe care o stim aproximativ constanta. Puteam alege si alte repartii deoarece nu avem detalii despre cereri, cat timp ne asiguram ca acestea sunt marginite.

Am ales lambda un numar care sa fie mai mare astfel incat simularile sa fie mai apropiate de o situatie mai realistica (i.e. 1000 de accesări în mediu per zi al resursei online).

Numarul de zile ales este aleator, dar din nou dorim un numar mai mare pentru a avea mai multe date deci vom alege un an.

```
lambda <- 1000  
nr_zile <- 365
```

```
total <- rpois(nr_zile, lambda)
head(total)
```

```
[1] 1009 1031 1034 958 986 985
```

Cerinta 3

Vom avea 3 probabilitati definite

```
p      <- 0.01
p2     <- 0.05
p3     <- 0.2
```

Cerinta 4

a) Verificare aleatoare simpla

Pentru testarea și aplicarea celor trei metode de verificare, am ales sa folosim repartizarea hypergeometrica deoarece definitia ei se potriveste pentru situatia noastra unde avem n cereri verificate din N cereri totale, stiind ca k dintre acestea sunt suspecte.

```
proc_verificare <- 10 / 100 #PARAMETRIZABIL
date$verificate_s <- floor(date$total * proc_verificare)
date$detectate_s <- rhyper(
  nn <- nr_zile,
  m = date$sus, # N2
  n = date$normale, #N1
  k = date$verificate_s # actual nr verificate
)
date$nedetectate_s <- date$sus - date$detectate_s

head(date$detectate_s)
```

```
[1] 0 1 1 2 0 1
```

```
head(date$nedetectate_s)
```

```
[1] 10 9 9 15 7 4
```

b) Verificare adaptiva 1

Crestem procentul odata ce vedem ca numarul de cereri din acea zi este mai mare decat media lambda

```
proc_adapt <- 20 / 100 + abs(date$total - lambda) * 80/100
date$verificate_a <- floor(date$total * proc_adapt / 100)

date$detectate_a <- rhyper(
  nn <- nr_zile,
  m = date$sus, # N2
  n = date$normale, #N1
  k = date$verificate_a # actual nr verificate
)
date$nedetectate_a <- date$sus - date$detectate_a

head(date$detectate_a)
```

```
[1] 0 3 2 6 3 0
```

```
head(date$nedetectate_a)
```

```
[1] 10 7 8 11 4 5
```

c) Verificare adaptiva 2

Procedam ca la punctul b doar ca de aceasta data, incercam ceva si mai agresiv care sa identifice mai multe cereri suspecte.

```
proc_adapt2 <- (20 + abs(date$total - lambda) * 0.4) / 100
date$verificate_a2 <- floor(date$total * proc_adapt2)
```

```
date$detectate_a2 <- rhyper(
  nn <- nr_zile,
  m = date$sus, # N2
  n = date$normale, #N1
  k = date$verificate_a2 # actual nr verificate
)
date$nedetectate_a2 <- date$sus - date$detectate_a2
```

```
head(date$detectate_a2)
```

```
[1] 1 3 3 6 5 3
```

```
head(date$nedetectate_a2)
```

```
[1] 9 7 7 11 2 2
```

d) Verificare adaptiva bazata pe Z-score (3)

Problema unei probabilitati constante (in cazul nostru 0.01, 0.05 si 0.2) si dorinta minimizarii costului de verificare si ratare a unei cereri suspecte duce la urmatoarele constatarari:

- Daca $p \cdot C_{\text{ratare}} > C_{\text{verificare}}$, strategia optima este sa verifichi toate cererile
- Daca $p \cdot C_{\text{ratare}} < C_{\text{verificare}}$, strategia optima este sa nu verifichi nimic.

Pentru aceasta strategie de verificare am incercat sa folosim o functie Sigmoida aplicata pe un scor Z (cate deviatii standard fata de medie are traficul dintr-o zi respectiva). Aceasta tine procentul de verificare mic, adica 10%, in zilele normale, dar va deveni “agresiva” si va creste spre procente mai mari (maxim 85%) pe spike-uri.

Functia logistica standard (sau sigmoidea) este de forma:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Adaptata problemei noastre aceasta ia urmatoarea forma:

$$f(x) = \text{procent baza} + \frac{\text{procent maxim} - \text{procent baza}}{1 + e^{-k(x-x_0)}}$$

Unde:

- procent_baza = 10% din cereri minimul, asimptota inferioara a functiei sigmoide
- procent_maxim = 85% din cereri, asimptota superioara a functiei sigmoide
- k = parametrul de “steepness”, care determina cat de abrupta este trecerea din starea normala in starea de alerta, adica detectarea spike-urilor.

- x = metrica pe care o evaluăm, adică Z-score
- x_0 = punctul de inflexiune

Pentru cazul nostru, vom considera $k = 2$ și inflexiunea să fie la 0.

```
base_proc <- 10 / 100
max_proc <- 85 / 100
Z_prag <- 0
steepness <- 2

deviatie_std <- sqrt(lambda)
date$Z_score <- (date$total - lambda) / deviatie_std

proc_risk <- base_proc + (max_proc - base_proc) / (1 + exp(-steepness * (date$Z_score - Z_prag)))

date$verificate_a3 <- floor(date$total * proc_risk)

date$verificate_a3 <- pmin(date$verificate_a3, date$total)

date$detectate_a3 <- rhyper(
  nn <- nr_zile,
  m = date$sus,
  n = date$normale,
  k = date$verificate_a3
)
date$nedetectate_a3 <- date$sus - date$detectate_a3
head(date$nedetectate_a3)
```

```
[1] 8 5 9 3 1 2
```

Cerinta 5

- Probabilitatea empirică de a detecta cel puțin o cerere suspectă într-o zi

```
vector_logic <- date$detectate_s > 0
prob_empiric <- sum(vector_logic) / length(vector_logic)
prob_empiric
```

```
[1] 0.6465753
```

- Proportia medie de cereri suspecte detectate

```
mean(date$detectate_s / date$sus)
```

```
[1] 0.09796406
```

- Proportia medie de cereri suspecte nedetectate

```
mean(date$nedetectate_s / date$sus)
```

```
[1] 0.9020359
```

- Numărul mediu de verificări efectuate zilnic

```
mean(date$verificate_s)
```

```
[1] 99.5589
```

- Un indicator de eficiență, ales și definit de echipă

Aici am ales ca indicator de eficienta media de cereri suspecte detectate dintre cele verificate.

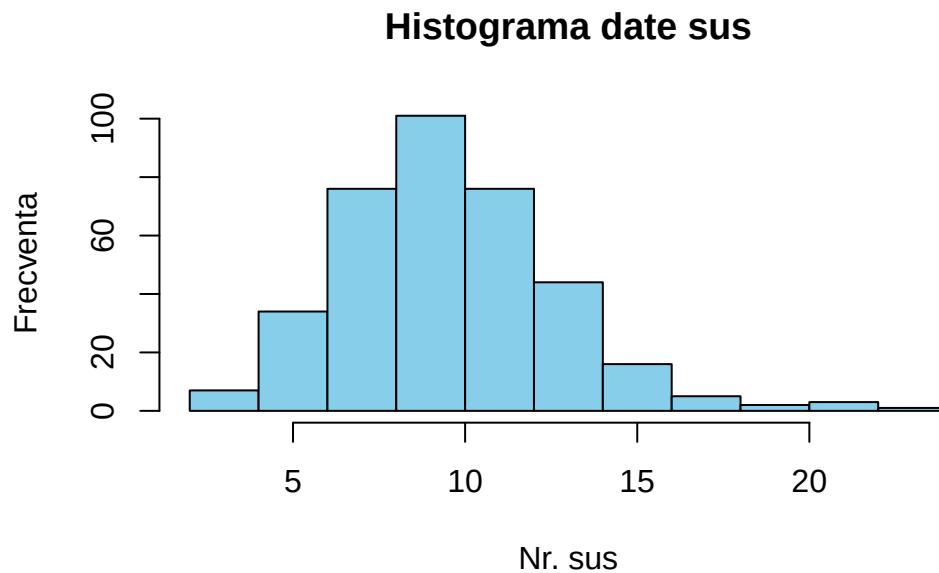
```
mean(date$detectate_s) / mean(date$verificate_s) * 100
```

```
[1] 1.009934
```

Cerinta 6

- Histograma numărului de cereri suspecte pe zi

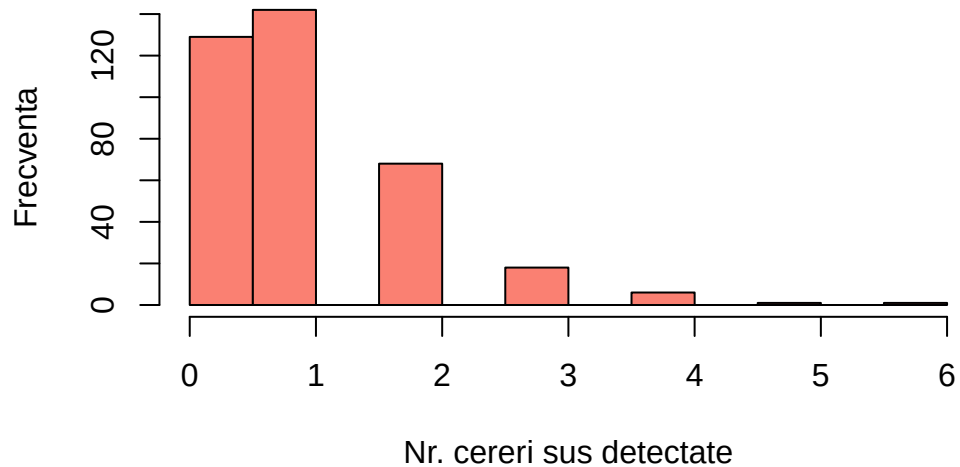
```
hist(date$sus,  
      col = "skyblue",  
      border = "black",  
      main = "Histograma date sus",  
      xlab = "Nr. sus",  
      ylab = "Frecventa",  
      breaks = 10  
    )
```



- Histograma numărului de cereri suspecte detectate

```
hist(date$detectate_s,  
      col = "salmon",  
      border = "black",  
      main = "Histograma cereri sus detectate per zi",  
      xlab = "Nr. cereri sus detectate",  
      ylab = "Frecventa",  
      breaks = 10  
    )
```

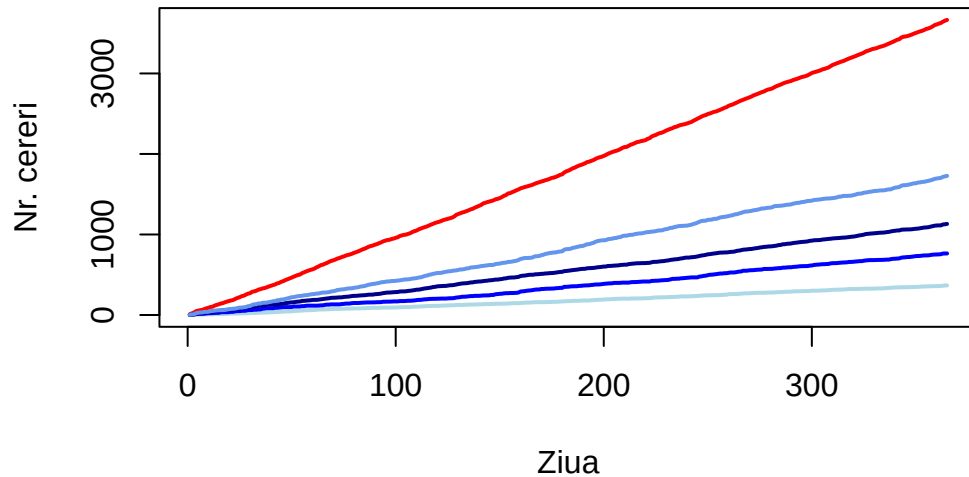
Histograma cereri sus detectate per zi



- Grafic comparativ între strategiile de verificare

```
plot(date$zi, cumsum(date$sus), type = "l", col = "red", lwd = 2,
      main = "Grafic comparativ între strategiile de verificare",
      xlab = "Ziua",
      ylab = "Nr. cereri",
      ylim = c(0, max(cumsum(date$sus))))
)
lines(date$zi, cumsum(date$detectate_s), col = "lightblue", lwd = 2)
lines(date$zi, cumsum(date$detectate_a), col = "blue", lwd = 2)
lines(date$zi, cumsum(date$detectate_a2), col = "darkblue", lwd = 2)
lines(date$zi, cumsum(date$detectate_a3), col = "cornflowerblue", lwd = 2)
```


Grafic comparativ între strategiile de verificare

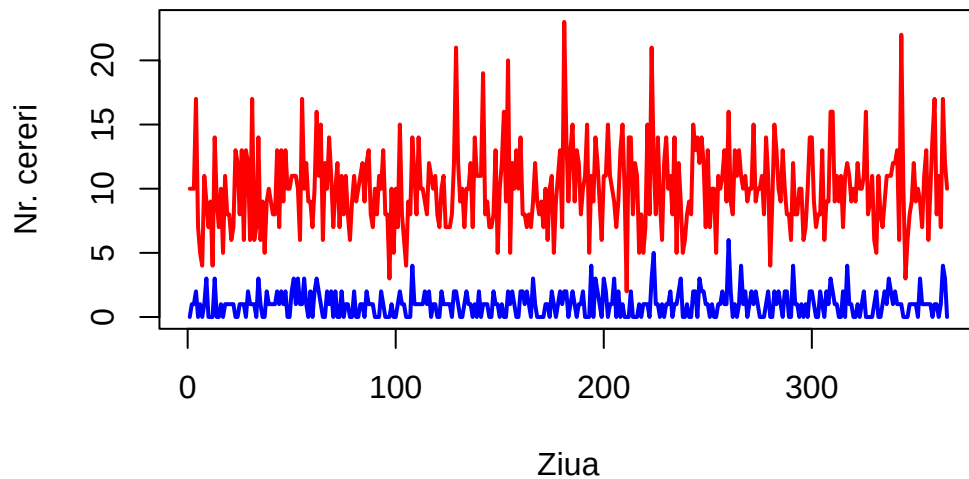


- Evoluția zilnică a cererilor suspecte și a celor detectate

Am facut un plot pentru zile impartit la numarul decereri:

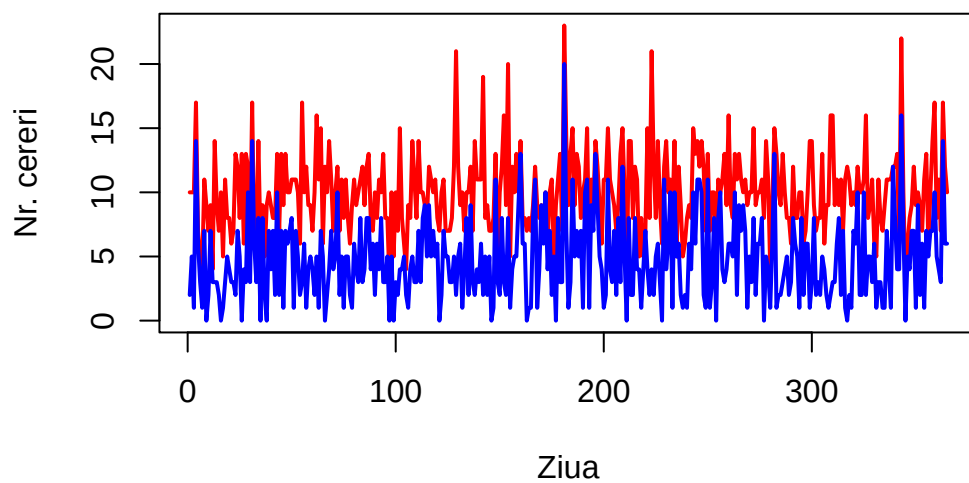
```
plot(date$zi, date$sus, type = "l", col = "red", lwd = 2,
      main = "Evolutia zilnica a cererilor sus si detectate,ver. simpla",
      xlab = "Ziua",
      ylab = "Nr. cereri",
      ylim = c(0, max(date$sus))
)
lines(date$zi, date$detectate_s, col = "blue", lwd = 2)
```

Evolutia zilnica a cererilor sus si detectate,ver. simpla



```
plot(date$zi, date$sus, type = "l", col = "red", lwd = 2,  
      main = "Evolutia zilnica a cererilor sus si detectate,ver. adaptiva 3",  
      xlab = "Ziua",  
      ylab = "Nr. cereri",  
      ylim = c(0, max(date$sus))  
    )  
lines(date$zi, date$detectate_a3, col = "blue", lwd = 2)
```

Evolutia zilnica a cererilor sus si detectate,ver. adaptiva :



Cerinta 7

```
val_sim <- vector("list", nr_zile)

for(i in 1:nr_zile) {
  val_sim[[i]] <- numeric(date$normale[i])
  vector_curent <- val_sim[[i]]
  indici_aleatori <- sample(1:date$normale[i], date$sus[i])
  vector_curent[indici_aleatori] <- 1
  val_sim[[i]] <- vector_curent
}

Calculare_Procent <- function(procent) {
  procent <- procent/100
  prob <- vector("numeric", nr_zile)
  for(i in 1:nr_zile){
    max_range <- date$normale[i]
    num_indexes <- round(date$normale[i]*procent, 1)
    distinct_indexes <- sample(1:max_range, num_indexes)
    prob[i] <- sum(val_sim[[i]][distinct_indexes])/date$sus[i]*100
  }
  return(mean(prob))
}

Repetare_Detect_Sus <- function(numar){
  Medie_y <- c(0,0,0,0,0)
  for(i in 1:numar){
    Medie_y <- Medie_y + c(
      Calculare_Procent(1),
      Calculare_Procent(5),
      Calculare_Procent(10),
      Calculare_Procent(20),
      Calculare_Procent(30)
    )
  }
  Medie_y <- Medie_y / numar
  return(Medie_y)
}

x <- c(1,5,10,20,30)
y <- c(
  Calculare_Procent(1),
  Calculare_Procent(5),
  Calculare_Procent(10),
  Calculare_Procent(20),
  Calculare_Procent(30)
)
head(y)
```

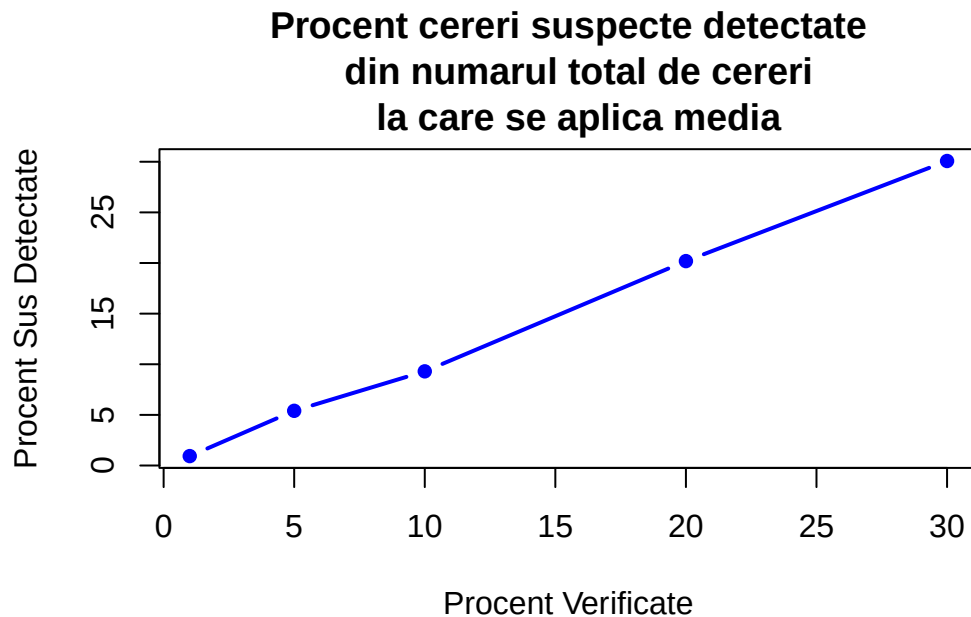
```
[1] 0.9292711 5.4034378 9.3008344 20.1876307 30.0755174
```

```
plot(x, y, type = "b", col = "blue", pch = 16, lwd = 2,
     main = "Procent cereri suspecte detectate")
```

```

din numarul total de cereri
la care se aplica media",
xlab = "Procent Verificate",
ylab = "Procent Sus Detectate"
)

```



```

y_100 <- Repetare_Detect_Sus(100)
head(y_100)

```

```

[1] 0.9336465 4.9215624 9.9719704 19.8914839 29.8930717

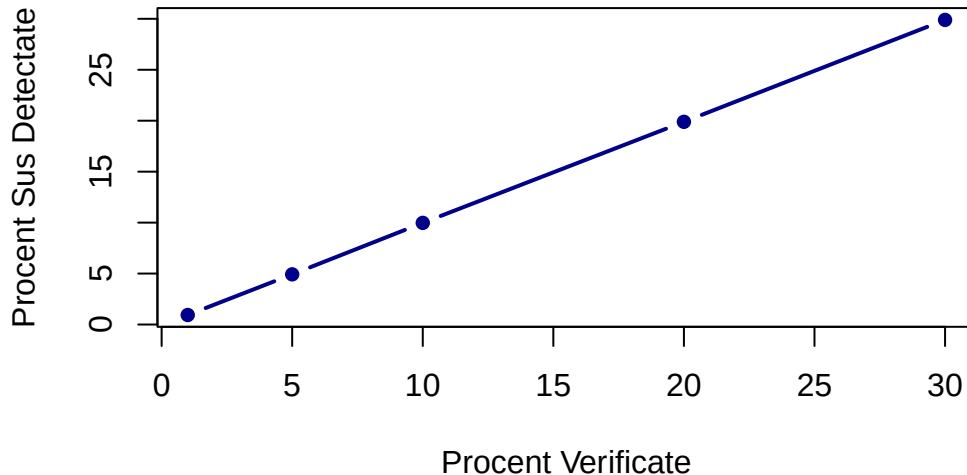
```

```

plot(x, y_100, type = "b", col = "darkblue", pch = 16, lwd = 2,
     main = "Procent sus detectate din sus total (media toate zilele)",
     xlab = "Procent Verificate",
     ylab = "Procent Sus Detectate"
)

```

Procent sus detectate din sus total (media toate zilele)



Se observa cum prin incercare aleatorie procentul de cereri suspicioase detectate creste linear cu procentul de date verificate dintre cele totale, deci probabilitatea de detectie se schimba linear.

Cerinta 8

In concluzie, se observa cum abordarea simpla aleatorie nu este destul pentru a detecta majoritatea cererilor suspicioase iar o abordare dinamica este necesara pentru a gasi mai multe cereri suspicioase in zilele cu mai multe cereri in general, iar pentru zilele cu mai putine cereri nu avem de ce sa cautam atat de multe cereri suspicioase deoarece nu sunt atat de mult de gasit. Strategia simpla este buna in sensul ca gaseste consistent un numar de cereri suspicioase pe zi raportat la numarul de cereri totale dar daca vrem sa maximizam numarul de cereri suspicioase gasite in raport cu numarul de incercari, atunci abordarea simpla pica. Abordarile dinamice au dus la reducerea costului implicat in verificari deoarece fiecare strategie a incercat sa reactioneze la schimbari fie in raport de medie si depasari ale acesteia fie prin adaptari la spike-uri de cereri suspicioase, strategia optima fiind cea care a utilizat functia sigmoida, care a reusit sa creasca exponential numarul de verificari necesare pe zi in functie de numarul de cereri suspicioase.

Cerinta ulterioara

Cerinta 1

```
c_verif <- 1
c_nedetct <- 1000
nr_repet <- 1000
CF_s <- c_verif * date$verificate_s + c_nedetct * date$nedetectate_s
mean(CF_s)
```

```
[1] 9135.175
```

```
CF_a <- c_verif * date$verificate_a + c_nedetct * date$nedetectate_a
mean(CF_a)
```

```
[1] 8152.888
```

```
CF_a2 <- c_verif * date$verificate_a2 + c_nedetect * date$nedetectate_a2  
mean(CF_a2)
```

```
[1] 7243.658
```

```
CF_a3 <- c_verif * date$verificate_a3 + c_nedetect * date$nedetectate_a3  
mean(CF_a3)
```

```
[1] 5786.047
```

Cerinta 2

```
costuri_s <- numeric(nr_repet)  
costuri_a <- numeric(nr_repet)  
costuri_a2 <- numeric(nr_repet)  
costuri_a3 <- numeric(nr_repet)  
  
for(i in 1:nr_repet){  
  set.seed(i)  
  date$detectate_s <- rhyper(nn <- nr_zile, m = date$sus, n = date$normale, k = date$verificate_s)  
  date$nedetectate_s <- date$sus - date$detectate_s  
  costuri_s[i] <- mean(c_verif * date$verificate_s + c_nedetect * date$nedetectate_s)  
  
  date$detectate_a <- rhyper(nn <- nr_zile, m = date$sus, n = date$normale, k = date$verificate_a)  
  date$nedetectate_a <- date$sus - date$detectate_a  
  costuri_a[i] <- mean(c_verif * date$verificate_a + c_nedetect * date$nedetectate_a)  
  
  date$detectate_a2 <- rhyper(nn <- nr_zile, m = date$sus, n = date$normale, k = date$verificate_a2)  
  date$nedetectate_a2 <- date$sus - date$detectate_a2  
  costuri_a2[i] <- mean(c_verif * date$verificate_a2 + c_nedetect * date$nedetectate_a2)  
  
  date$detectate_a3 <- rhyper(nn <- nr_zile, m = date$sus, n = date$normale, k = date$verificate_a3)  
  date$nedetectate_a3 <- date$sus - date$detectate_a3  
  costuri_a3[i] <- mean(c_verif * date$verificate_a3 + c_nedetect * date$nedetectate_a3)  
}  
  
head(costuri_s)
```

```
[1] 9187.230 9176.271 9154.353 9140.655 9157.093 9039.285
```

```
head(costuri_a)
```

```
[1] 8078.915 8180.285 8098.093 8174.805 8158.367 8213.162
```

```
head(costuri_a2)
```

```
[1] 7320.370 7347.767 7323.110 7460.096 7238.178 7377.904
```

```
head(costuri_a3)
```

```
[1] 5777.827 5695.636 5731.252 5750.430 5720.293 5676.458
```

```
mean(costuri_s)
```

```
[1] 9140.907
```

```
mean(costuri_a)
```

```
[1] 8191.225
```

```
mean(costuri_a2)
```

```
[1] 7314.586
```

```
mean(costuri_a3)
```

```
[1] 5773.973
```

```
sd(costuri_s)
```

```
[1] 48.08774
```

```
sd(costuri_a)
```

```
[1] 61.78477
```

```
sd(costuri_a2)
```

```
[1] 73.9107
```

```
sd(costuri_a3)
```

```
[1] 69.40625
```

Cerinta 3

Probabilitatiile teoretice sunt valorile exacte obtinute prin demonstratie matematica, folosind teoreme si functii de masa sau densitate de probabilitate. Probabilitatiile simulate sunt valori aproximative obtinute prin simularea unui model de un numar mare de ori si extragerea unori statistici cum ar fi media sau deviata standard din setul de date.

Documentatie Problema 2

In realizarea aplicatiei Shiny am folosit urmatoarele pachete: shiny, ggplot2, shinythemes pentru dark-theme si MASS pentru functia mvrnorm.

Cerinta obligatorie

Cerinta 1

Poza cu structura aplicatiei si privire din ansamblu.

Transformări de Variabile Aleatoare Continue

Dimensiunea globală a Eșantionului ($n \geq 10$):

Opțiuni

Alege Repartiția lui X:

Normală (Gaussiană)

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Medie (μ):

Deviație Standard ($\sigma > 0$):

Alege Transformarea $Y = g(X)$:

$g(x) = x^2$

Generează / Actualizează

Componenta Unidimensională Componenta bidimensională

Distribuția Originală X

Eroare: Deviația standard trebuie să fie strict pozitivă!

Distribuția Transformată Y

Eroare: Deviația standard trebuie să fie strict pozitivă!

Indicatori Statistici pentru X

Eroare: Deviația standard trebuie să fie strict pozitivă!

Indicatori Statistici pentru Y

Eroare: Deviația standard trebuie să fie strict pozitivă!

Interpretare automată:

Eroare: Deviația standard trebuie să fie strict pozitivă!

Cerinta 2

Repartitia normala:

Alege Repartiția lui X:

Normală (Gaussiană)

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Medie (μ):

0

Deviație Standard ($\sigma > 0$):

1

Repartitia uniforma:

Alege Repartiția lui X:

Uniformă

$$f(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{altfel} \end{cases}$$

Interval $[a, b]$:

-100 -80 -60 -40 -20 0 20 40 60 80 100

-2 -2

Repartitia exponentiala:

Alege Repartiția lui X:

Exponențială

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & \text{altfel} \end{cases}$$

Rată ($\lambda > 0$):

1

Repartitia gamma:

Alege Repartiția lui X:

Gamma

$$f(x) = \begin{cases} \frac{\lambda^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\lambda x} & x > 0 \\ 0 & \text{altfel} \end{cases}$$

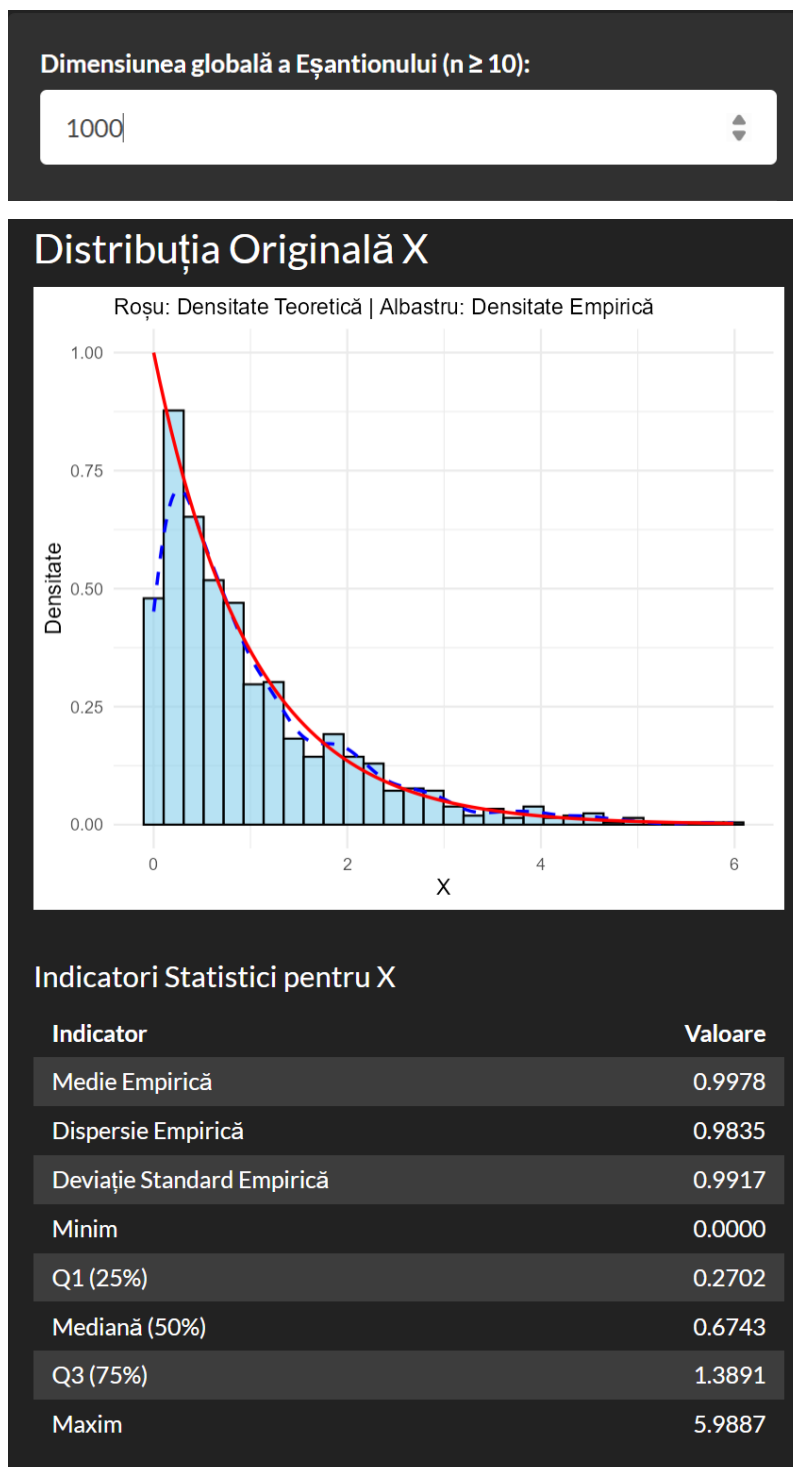
Formă ($\alpha > 0$):

2

Rată ($\lambda > 0$):

1

Cerinta 3



Cerinta 4

Alege Transformarea $Y = g(X)$:

$g(x) = x^2$

$g(x) = x^2$

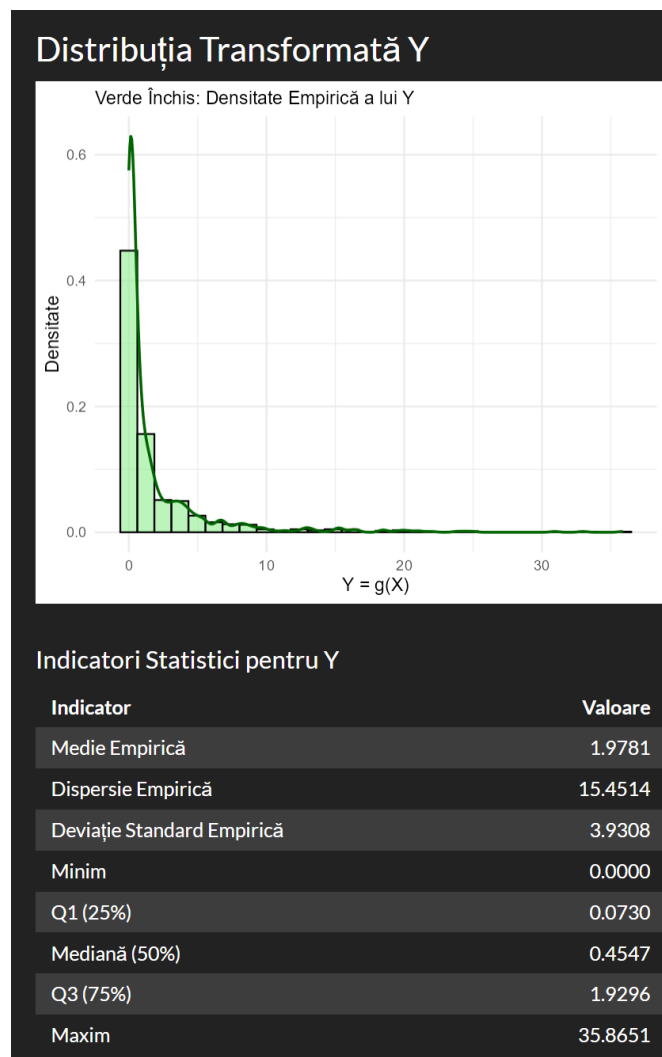
$g(x) = |x|$

$g(x) = \log(x)$

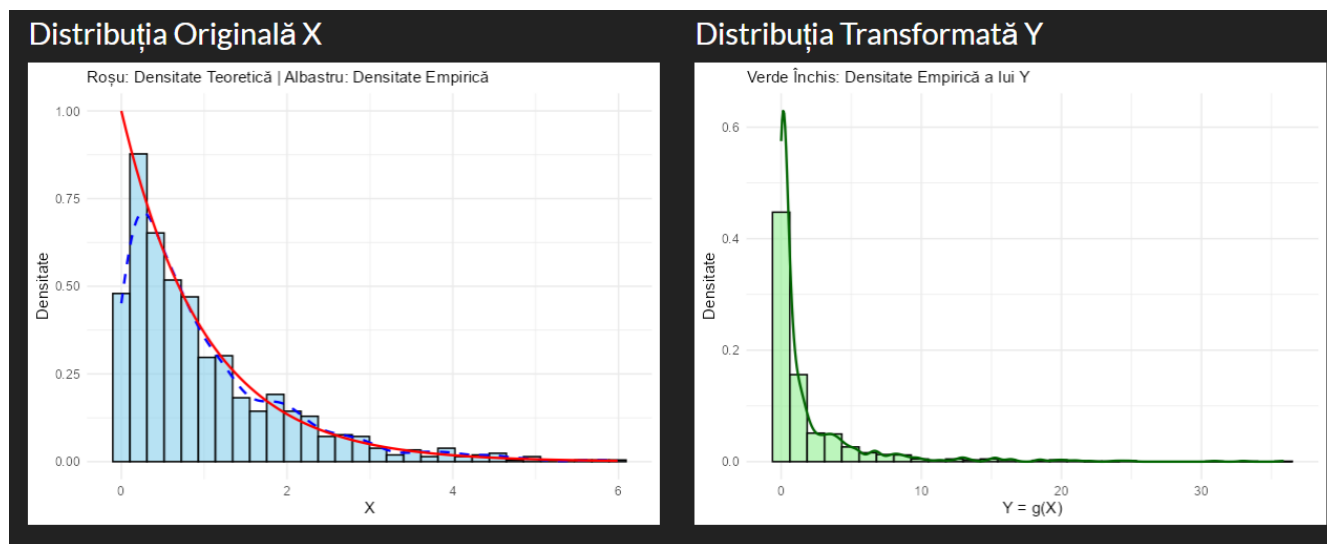
$g(x) = e^x$

$g(x) = \sin(x)$

Cerinta 5



Cerinta 6



Cerinta 7

Interpretare automată:	
Indicator	Valoare
Distribuția X	asimetrică pozitiv
Distribuția Y	asimetrică pozitiv
Transformarea a modificat simetria	FALSE
Transformarea a influențat valorile mari prin	extindere
Transformarea a produs	valori strict pozitive
Transformarea a influențat valorile extreme prin	accentuare

##TO:DO CA NU MAI AM CHEF. ## EXPLICATI CUM SE GENEREAZA TABELUL ASTA TOTU E IN CODU DE program1.r la functia calc_auto. Adica ce inseamna acel skewness (Pearson stuff din sursa), restul is cam ez de explicat.

Cerinta 8

Vom considera $X \sim N(0, 1)$ si transformările $Y = \log X$ si $Y = e^X$.

Transformarea $Y = e^X$ transforma valorile negative ale lui X in valori subunitare ($0 < e^X < 1$), cele pozitive dau valori supraunitare. In acest caz, Y va fi mereu pozitiva, asimetrica spre dreapta. Valorile mari sunt rare dar pot fi foarte mari.

Transformarea $Y = \log(X)$ este problematica deoarece $\log(x)$ nu este definita pentru valori mai mici sau egale cu 0. Deoarece X ia valori negative cu o probabilitate de 50%, aceasta transformare este invalida pe jumatate din domeniu. Aplicatia noastra elimina in acest caz valorile care sunt fie NaN, fie infinite.

Componentă bidimensională obligatorie

Cerinta 1

Parametrii Simulării 2D

Modul de generare (X, Y):

☒ Independente

☐ Normală Bidimensională

Cerinta 2

Cerinta 3

Cerinta 4

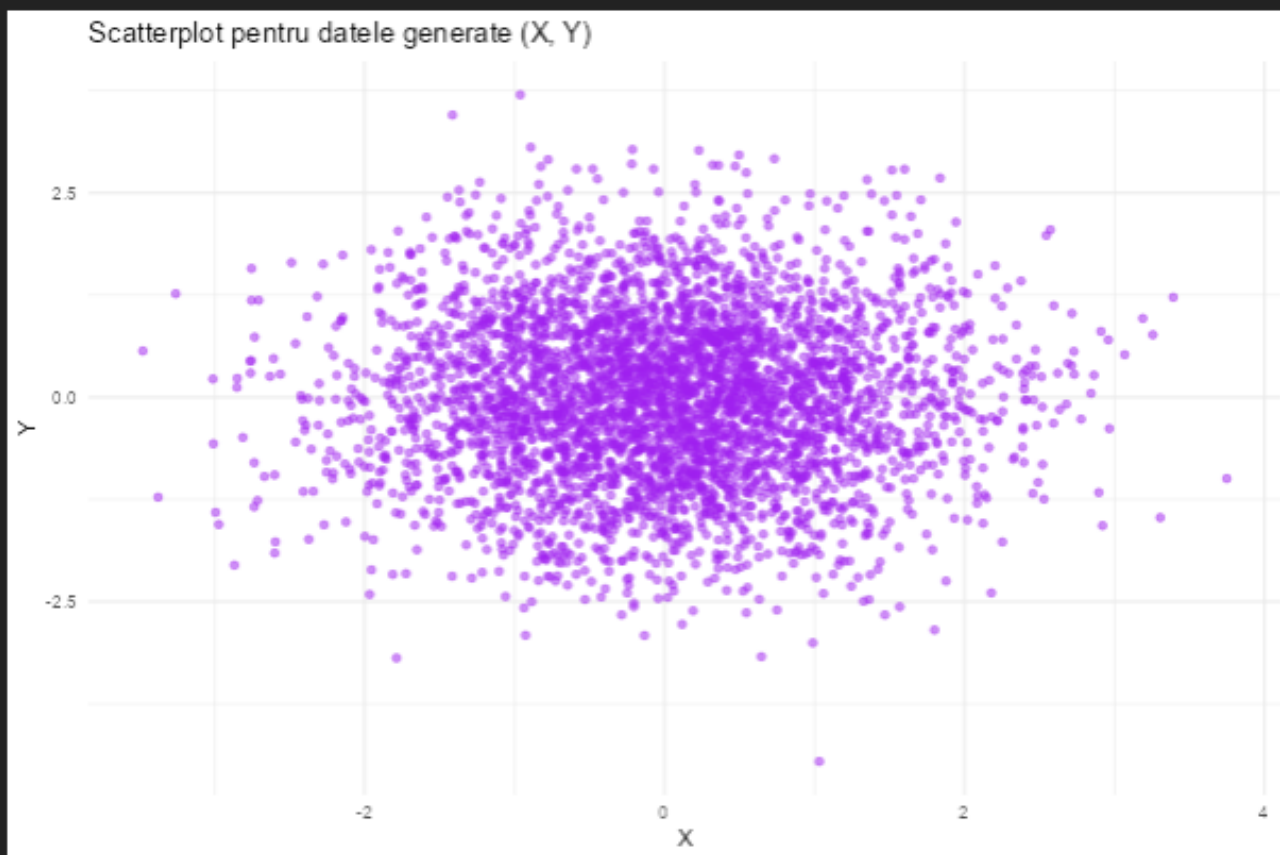
Cerinta 5

Cerinta 6

Parametrul de corelație are un efect direct, liniar și proporțional asupra covarianței. Având abaterile standard σ_X și σ_Y constante, influența lui ρ este următoarea:

- Dacă $\rho = 0$ covarianța tinde spre 0, indiferent de mărimea abaterilor standard. Cele două variabile nu prezintă o tendință liniară de a crește sau scădește împreună. Norul de puncte este circular sau o elipsă ușoară.

Scatterplot Perechi (X, Y)



Indicator

Valoare

Covarianță Empirică (X,Y)

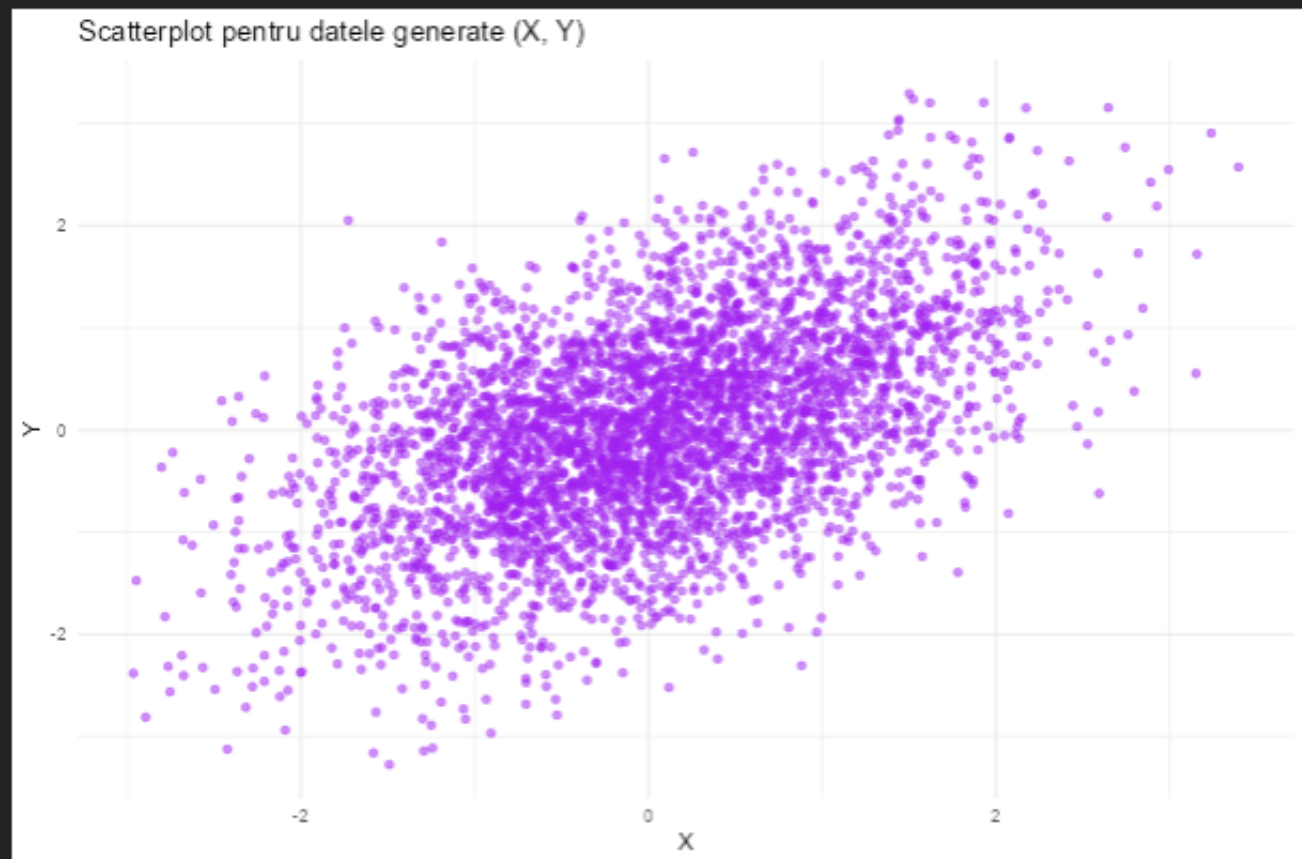
0.00676354417182671

Coeficient de Corelație (X,Y)

0.00665417641081049

- Dacă $\rho = 0.5$ covarianța devine pozitivă. Creșterea parametrului mărește direct proporțional covarianța. Norul de puncte ia o formă elipsoidală spre dreapta-sus, indicând că valorile mari ale lui X tind să fie asociate cu valori mari ale lui Y.

Scatterplot Perechi (X, Y)



Indicator	Valoare
Covarianță Empirică (X,Y)	0.505558203360397
Coeficient de Corelație (X,Y)	0.515467657019738

- Dacă $\rho = -0.5$ covarianța devine negativă. Pe măsura ce parametrul se apropie de -1, covarianța scade. Norul de puncte ia o formă elipsoidală și se înclină spre dreapta-jos, indicând că valorile mari ale lui X sunt asociate cu valori mici ale lui Y.

Scatterplot Perechi (X, Y)



Indicator	Valoare
Covarianță Empirică (X,Y)	-0.508240150659652
Coeficient de Corelație (X,Y)	-0.508527831705237